

	Опис ґраматики за допомогою РБНФ	Опис ґраматика (правила)	LL2-нотація для правил ґраматикиОпис ґраматика		Код для перевірки ґраматики заданої за допомогою РБНФ	/* Дані для перевірки перевірки ґраматики за допомогою LL2-аналізатора УВАГА: при копіюванні зважайте, щоб у кожному рядку після символу «\» не містилось жодних інших символів. */
РБНФ				Код для перевірки РБНФ		
		G = (N, T, P, S) N = { labeled_point ident goto_label program_name value_type array_specify declaration_element array_specify_optional other_declaration_ident declaration other_declaration_ident__iteration index_action unary_operator unary_operation binary_operator binary_action left_expression group_expression value index_action__optional expression binary_action__iteration expression_or_bind_left_to_right bind_to_right bind_to_right__optional if_expression body_for_true false_cond_block_without_else body_for_false cond_block false_cond_block_without_else__iteration body_for_false__optional cond_block__with_optional_bind cycle_begin_expression cycle_end_expression cycle_counter cycle_counter_lr_init cycle_counter_init cycle_counter_last_value cycle_body forto_direction forto_cycle continue_while break_while statement_in_while_and_if_body statement block_statements_in_while_and_if_body statement_in_while_and_if_body__iteration while_cycle_head_expression while_cycle repeat_until_cycle_cond repeat_until_cycle statements_or_block_statements block_statements input_rule argument_for_input output_rule statement__iteration expression__optional program_rule declaration__optional unsigned_value value sign_optional sign ident sign_plus sign_minus }		G = (N, T, P, S) N = { labeled_point ident goto_label program_name value_type array_specify declaration_element array_specify_optional other_declaration_ident declaration other_declaration_ident__iteration index_action unary_operator unary_operation binary_operator binary_action left_expression group_expression value index_action__optional expression binary_action__iteration expression_or_bind_left_to_right bind_to_right bind_to_right__optional if_expression body_for_true false_cond_block_without_else body_for_false cond_block false_cond_block_without_else__iteration body_for_false__optional cond_block__with_optional_bind cycle_begin_expression cycle_end_expression cycle_counter cycle_counter_lr_init cycle_counter_init cycle_counter_last_value cycle_body forto_direction forto_cycle continue_while break_while statement_in_while_and_if_body statement block_statements_in_while_and_if_body statement_in_while_and_if_body__iteration while_cycle_head_expression while_cycle repeat_until_cycle_cond repeat_until_cycle statements_or_block_statements block_statements input_rule argument_for_input output_rule statement__iteration expression__optional program_rule declaration__optional unsigned_value value sign_optional sign ident sign_plus sign_minus }		
		T = { ".", ",", "GOTO", "INTEGER16", " ", "NOT", "AND", "OR", "=", "!=", "<=", ">=", "<", ">", "+", "-", "*", "/", "DIV", "MOD", "(", ")", ";", "ELSE", "IF", "DO", "FOR", "TO", "DOWNT", "WHILE", "CONTINUE", "BREAK", }		T = { ".", ",", "GOTO", "INTEGER16", " ", "NOT", "AND", "OR", "=", "!=", "<=", ">=", "<", ">", "+", "-", "*", "/", "DIV", "MOD", "(", ")", ";", "ELSE", "IF", "DO", "FOR", "TO", "DOWNT", "WHILE", "CONTINUE", "BREAK", }		

		<pre>"EXIT", "REPEAT", "UNTIL", "GET", "PUT", "NAME", "BODY", "DATA", "BEGIN", "END", "(", ")", "[", "]", ";", ",", "=", "+", "ident_terminal", "unsigned_value_terminal" }</pre>		<pre>"EXIT", "REPEAT", "UNTIL", "GET", "PUT", "NAME", "BODY", "DATA", "BEGIN", "END", "(", ")", "[", "]", ";", ",", "=", "+", "ident_terminal", "unsigned_value_terminal" }</pre>		
				tokenCOLON = ":" >> BOUNDARIES;		<pre>#define T_COLON_0 ":" #define T_COLON_1 "" #define T_COLON_2 "" #define T_COLON_3 ""</pre>
				tokenGOTO = "GOTO" >> STRICT_BOUNDARIES;		<pre>#define T_GOTO_0 "GOTO" #define T_GOTO_1 "" #define T_GOTO_2 "" #define T_GOTO_3 ""</pre>
				tokenINTEGER16 = "INTEGER16" >> STRICT_BOUNDARIES;		<pre>#define T_DATA_TYPE_0 "INTEGER16" #define T_DATA_TYPE_1 "" #define T_DATA_TYPE_2 "" #define T_DATA_TYPE_3 ""</pre>
				tokenCOMMA = "," >> BOUNDARIES;		<pre>#define T_COMA_0 "," #define T_COMA_1 "" #define T_COMA_2 "" #define T_COMA_3 ""</pre>
						<pre>#define T_BITWISE_NOT_0 "~" #define T_BITWISE_NOT_1 "" #define T_BITWISE_NOT_2 "" #define T_BITWISE_NOT_3 ""</pre>
				tokenNOT = "NOT" >> STRICT_BOUNDARIES;		<pre>#define T_NOT_0 "NOT" #define T_NOT_1 "" #define T_NOT_2 "" #define T_NOT_3 ""</pre>
						<pre>#define T_BITWISE_AND_0 "&" #define T_BITWISE_AND_1 "" #define T_BITWISE_AND_2 "" #define T_BITWISE_AND_3 ""</pre>
				tokenAND = "AND" >> STRICT_BOUNDARIES;		<pre>#define T_AND_0 "AND" #define T_AND_1 "" #define T_AND_2 "" #define T_AND_3 ""</pre>
						<pre>#define T_BITWISE_OR_0 " " #define T_BITWISE_OR_1 "" #define T_BITWISE_OR_2 "" #define T_BITWISE_OR_3 ""</pre>
				tokenOR = "OR" >> STRICT_BOUNDARIES;		<pre>#define T_OR_0 "OR" #define T_OR_1 "" #define T_OR_2 "" #define T_OR_3 ""</pre>
				tokenEQUAL = "==" >> BOUNDARIES;		<pre>#define T_EQUAL_0 "==" #define T_EQUAL_1 "" #define T_EQUAL_2 "" #define T_EQUAL_3 ""</pre>
				tokenNOTEQUAL = "!=" >> BOUNDARIES;		<pre>#define T_NOT_EQUAL_0 "!=" #define T_NOT_EQUAL_1 "" #define T_NOT_EQUAL_2 "" #define T_NOT_EQUAL_3 ""</pre>
				tokenLESSOREQUAL = "<=" >> BOUNDARIES;		<pre>#define T_LESS_OR_EQUAL_0 "<=" #define T_LESS_OR_EQUAL_1 "" #define T_LESS_OR_EQUAL_2 "" #define T_LESS_OR_EQUAL_3 ""</pre>
				tokenGREATEROREQUAL = ">=" >> BOUNDARIES;		<pre>#define T_GREATER_OR_EQUAL_0 ">=" #define T_GREATER_OR_EQUAL_1 "" #define T_GREATER_OR_EQUAL_2 "" #define T_GREATER_OR_EQUAL_3 ""</pre>
				tokenLESS = "<" >> BOUNDARIES;		<pre>#define T_LESS_0 "<" #define T_LESS_1 "" #define T_LESS_2 "" #define T_LESS_3 ""</pre>
				tokenGREATER = ">" >> BOUNDARIES;		<pre>#define T_GREATER_0 ">" #define T_GREATER_1 "" #define T_GREATER_2 "" #define T_GREATER_3 ""</pre>
				tokenPLUS = "+" >> BOUNDARIES;		<pre>#define T_ADD_0 "+" #define T_ADD_1 "" #define T_ADD_2 "" #define T_ADD_3 ""</pre>
				tokenMINUS = "-" >> BOUNDARIES;		<pre>#define T_SUB_0 "-" #define T_SUB_1 "" #define T_SUB_2 "" #define T_SUB_3 ""</pre>
				tokenMUL = "*" >> BOUNDARIES;		<pre>#define T_MUL_0 "*" #define T_MUL_1 "" #define T_MUL_2 "" #define T_MUL_3 ""</pre>
				tokenDIV = "DIV" >> STRICT_BOUNDARIES;		<pre>#define T_DIV_0 "DIV" #define T_DIV_1 "" #define T_DIV_2 "" #define T_DIV_3 ""</pre>
				tokenMOD = "MOD" >> STRICT_BOUNDARIES;		<pre>#define T_MOD_0 "MOD" #define T_MOD_1 "" #define T_MOD_2 "" #define T_MOD_3 ""</pre>
				tokenGROUPEXPRESSIONBEGIN = "(" >> BOUNDARIES;		<pre>#define T_BEGIN GROUPEXPRESSION_0 "(" #define T_BEGIN GROUPEXPRESSION_1 ""</pre>

						#define T_BEGIN GROUPEXPRESSION_2 "" #define T_BEGIN GROUPEXPRESSION_3 ""
				tokenGROUPEXPRESSIONEND = "}" >> BOUNDARIES;		#define T_END GROUPEXPRESSION_0 ")" #define T_END GROUPEXPRESSION_1 "" #define T_END GROUPEXPRESSION_2 "" #define T_END GROUPEXPRESSION_3 ""
				tokenLRBIND = "=" >> BOUNDARIES;		#define T_LRBIND_0 "=: " #define T_LRBIND_1 "" #define T_LRBIND_2 "" #define T_LRBIND_3 ""
						#define T_THEN_BLOCK_0 "{ " #define T_THEN_BLOCK_1 "" #define T_THEN_BLOCK_2 "" #define T_THEN_BLOCK_3 ""
				tokenELSE = "ELSE" >> STRICT_BOUNDARIES;		#define T_ELSE_BLOCK_0 "ELSE " #define T_ELSE_BLOCK_1 "{ " #define T_ELSE_BLOCK_2 "" #define T_ELSE_BLOCK_3 ""
				tokenIF = "IF" >> STRICT_BOUNDARIES;		#define T_IF_0 "IF " #define T_IF_1 "" #define T_IF_2 "" #define T_IF_3 ""
						#define T_ELSE_IF_0 "ELSE " #define T_ELSE_IF_1 "IF " #define T_ELSE_IF_2 "" #define T_ELSE_IF_3 ""
				tokenDO = "DO" >> STRICT_BOUNDARIES;		#define T_DO_0 "DO " #define T_DO_1 "" #define T_DO_2 "" #define T_DO_3 ""
				tokenFOR = "FOR" >> STRICT_BOUNDARIES;		#define T_FOR_0 "FOR " #define T_FOR_1 "" #define T_FOR_2 "" #define T_FOR_3 ""
				tokenTO = "TO" >> STRICT_BOUNDARIES;		#define T_TO_0 "TO " #define T_TO_1 "" #define T_TO_2 "" #define T_TO_3 ""
				tokenDOWNTO = "DOWNTO" >> STRICT_BOUNDARIES;		#define T_DOWNTO_0 "DOWNTO " #define T_DOWNTO_1 "" #define T_DOWNTO_2 "" #define T_DOWNTO_3 ""
				tokenWHILE = "WHILE" >> STRICT_BOUNDARIES;		#define T_WHILE_0 "WHILE " #define T_WHILE_1 "" #define T_WHILE_2 "" #define T_WHILE_3 ""
				tokenCONTINUE = "CONTINUE" >> STRICT_BOUNDARIES;		#define T_CONTINUE_WHILE_0 "CONTINUE " #define T_CONTINUE_WHILE_1 "" #define T_CONTINUE_WHILE_2 "" #define T_CONTINUE_WHILE_3 ""
				tokenBREAK = "BREAK" >> STRICT_BOUNDARIES;		#define T_EXIT_WHILE_0 "BREAK " #define T_EXIT_WHILE_1 "" #define T_EXIT_WHILE_2 "" #define T_EXIT_WHILE_3 ""
				tokenEXIT = "EXIT" >> STRICT_BOUNDARIES;		#define T_EXIT_0 "EXIT " #define T_EXIT_1 "" #define T_EXIT_2 "" #define T_EXIT_3 ""
				tokenREPEAT = "REPEAT" >> STRICT_BOUNDARIES;		#define T_REPEAT_0 "REPEAT " #define T_REPEAT_1 "" #define T_REPEAT_2 "" #define T_REPEAT_3 ""
				tokenUNTIL = "UNTIL" >> STRICT_BOUNDARIES;		#define T_UNTIL_0 "UNTIL " #define T_UNTIL_1 "" #define T_UNTIL_2 "" #define T_UNTIL_3 ""
				tokenGET = "GET" >> STRICT_BOUNDARIES;		#define T_INPUT_0 "GET " #define T_INPUT_1 "" #define T_INPUT_2 "" #define T_INPUT_3 ""
				tokenPUT = "PUT" >> STRICT_BOUNDARIES;		#define T_OUTPUT_0 "PUT " #define T_OUTPUT_1 "" #define T_OUTPUT_2 "" #define T_OUTPUT_3 ""
				tokenNAME = "NAME" >> STRICT_BOUNDARIES;		#define T_NAME_0 "NAME " #define T_NAME_1 "" #define T_NAME_2 "" #define T_NAME_3 ""
				tokenBODY = "BODY" >> STRICT_BOUNDARIES;		#define T_BODY_0 "BODY " #define T_BODY_1 "" #define T_BODY_2 "" #define T_BODY_3 ""
				tokenDATA = "DATA" >> STRICT_BOUNDARIES;		#define T_DATA_0 "DATA " #define T_DATA_1 "" #define T_DATA_2 "" #define T_DATA_3 ""
				tokenBEGIN = "BEGIN" >> STRICT_BOUNDARIES;		#define T_BEGIN_0 "BEGIN " #define T_BEGIN_1 "" #define T_BEGIN_2 "" #define T_BEGIN_3 ""
				tokenEND = "END" >> STRICT_BOUNDARIES;		#define T_END_0 "END " #define T_END_1 "" #define T_END_2 "" #define T_END_3 ""
				tokenBEGINBLOCK = "{" >> BOUNDARIES;		#define T_BEGIN_BLOCK_0 "{ " #define T_BEGIN_BLOCK_1 "" #define T_BEGIN_BLOCK_2 "" #define T_BEGIN_BLOCK_3 ""
				tokenENDBLOCK = "}" >> BOUNDARIES;		#define T_END_BLOCK_0 "}" #define T_END_BLOCK_1 "" #define T_END_BLOCK_2 "" #define T_END_BLOCK_3 ""
				tokenLEFTSSQUAREBRACKETS = "[" >> BOUNDARIES;		#define T_LEFT_SQUAREBRACKETS_0 "[" #define T_LEFT_SQUAREBRACKETS_1 "" #define T_LEFT_SQUAREBRACKETS_2 "" #define T_LEFT_SQUAREBRACKETS_3 ""

				tokenRIGHTSSQUAREBRACKETS = "]" >> BOUNDARIES;		#define T_RIGHT_SQUAREBRACKETS_0 "]" #define T_RIGHT_SQUAREBRACKETS_1 "" #define T_RIGHT_SQUAREBRACKETS_2 "" #define T_RIGHT_SQUAREBRACKETS_3 ""
				tokenSEMICOLON = ";" >> BOUNDARIES;		#define T_SEMICOLON_0 ";" #define T_SEMICOLON_1 "" #define T_SEMICOLON_2 "" #define T_SEMICOLON_3 ""
						#define T_NULL_STATEMENT_0 "NULL" #define T_NULL_STATEMENT_1 "STATEMENT" #define T_NULL_STATEMENT_2 "" #define T_NULL_STATEMENT_3 ""
						#define GRAMMAR_LL2__2025 {\
labeled_point = ident , ":";				labeled_point = ident >> tokenCOLON;		{ LA_IS, ("ident_terminal"), { "labeled_point", {\
goto_label = "GOTO" , ident;				goto_label = tokenGOTO >> ident;		{ LA_IS, (T_GOTO_0), { "goto_label", {\
program_name = ident;				program_name = SAME_RULE(ident);		{ LA_IS, ("ident_terminal"), { "program_name", {\
value_type = "INTEGER16";				value_type = SAME_RULE(tokenINTEGER16);		{ LA_IS, (T_DATA_TYPE_0), { "value_type", {\
						{ LA_IS, ("["), { "array_specify", {\
declaration_element = ident , ["[" , unsigned_value , "]"];				declaration_element = ident >> -(tokenLEFTSSQUAREBRACKETS >> unsigned_value >> tokenRIGHTSSQUAREBRACKETS);		{ LA_IS, ("ident_terminal"), { "declaration_element", {\
						{ LA_IS, ("["), { "array_specify_optional", {\
other_declaration_ident = " , " , declaration_element;				other_declaration_ident = tokenCOMMA >> declaration_element;		{ LA_IS, (T_COMA_0), { "other_declaration_ident", {\
declaration = value_type , declaration_element , {other_declaration_ident};				declaration = value_type >> declaration_element >> *other_declaration_ident;		{ LA_IS, (T_DATA_TYPE_0), { "declaration", {\
						{ LA_IS, (T_COMA_0), { "other_declaration_ident__iteration", {\
index_action = "[" , expression , "]" ;				index_action = tokenLEFTSSQUAREBRACKETS >> expression >> tokenRIGHTSSQUAREBRACKETS;		{ LA_IS, ("["), { "index_action", {\
unary_operator = "NOT";				unary_operator = SAME_RULE(tokenNOT);		{ LA_IS, (T_NOT_0), { "unary_operator", {\
unary_operation = unary_operator , expression;				unary_operation = unary_operator >> expression;		{ LA_IS, (T_NOT_0), { "unary_operation", {\
binary_operator = "AND" "OR" "==" "!=" "<=" ">=" "<" ">" "+" "-" "*" "DIV" "MOD";				binary_operator = tokenAND tokenOR tokenEQUAL tokenNOTEQUAL tokenLESSOREQUAL tokenGREATEROREQUAL tokenLESS tokenGREATER tokenPLUS tokenMINUS tokenMUL tokenDIV tokenMOD;		{ LA_IS, (T_AND_0), { "binary_operator", {\
binary_action = binary_operator , expression;				binary_action = binary_operator >> expression;		IF_NONUSE_COMPARE_WITH_EQUAL{\

						<pre> { LA_IS, "", 2, { "binary_operator", "expression" }}\ }};\ }\ IF_USE_COMPARE_WITH_EQUAL(\ { LA_IS, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_OR_EQUAL_0, T_GREATER_OR_EQUAL_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, { "binary_action", (\ _ { LA_IS, "", 2, { "binary_operator", "expression" }}\ }};\ }\ </pre>
<pre> left_expression = group_expression unary_operation ident , [index_action] value cond_block; </pre>				<pre> left_expression = group_expression unary_operation ident >> -index_action value cond_block; </pre>		<pre> {LA_IS, { "(" , { "left_expression", (\ {LA_IS, { "" }, 1, { "group_expression" }}\ }};\ {LA_IS, { T_NOT_0 }, { "left_expression", (\ {LA_IS, { "" }, 1, { "unary_operation" }}\ }};\ {LA_IS, { T_ADD_0, T_SUB_0 }, { "left_expression", (\ {LA_IS, { "unsigned_value_terminal", 1, { "value" }}\ {LA_NOT, { "unsigned_value_terminal", 1, { "unary_operation" }}\ }};\ {LA_IS, { "ident_terminal" }, { "left_expression", (\ {LA_IS, { "" }, 2, { "ident", "index_action__optional" }}\ }};\ {LA_IS, { "unsigned_value_terminal" }, { "left_expression", (\ {LA_IS, { "" }, 1, { "value" }}\ }};\ {LA_IS, { "T_IF_0" }, { "left_expression", (\ {LA_IS, { "" }, 1, { "cond_block" }}\ }};\ }\ </pre>
						<pre> {LA_IS, { "[" , { "index_action__optional", (\ {LA_IS, { "" }, 1, { "index_action" }}\ }};\ {LA_NOT, { "[" , { "index_action__optional", (\ {LA_IS, { "" }, 0, { "" }}\ }};\ }\ </pre>
<pre> expression = left_expression , {binary_action}; </pre>				<pre> expression = left_expression >> *binary_action; </pre>		<pre> {LA_IS, { "(" , T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, { "expression", (\ {LA_IS, { "" }, 2, { "left_expression", "binary_action__iteration" }}\ }};\ }\ </pre>
						<pre> IF_NONUSE_COMPARE_WITH_EQUAL(\ {LA_IS, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_0, T_GREATER_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, { "binary_action__iteration", (\ _ {LA_IS, { "" }, 2, { "binary_action", "binary_action__iteration" }}\ }};\ {LA_NOT, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_0, T_GREATER_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, { "binary_action__iteration", (\ _ {LA_IS, { "" }, 0, { "" }}\ }};\ }\ IF_USE_COMPARE_WITH_EQUAL(\ {LA_IS, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_OR_EQUAL_0, T_GREATER_OR_EQUAL_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, { "binary_action__iteration", (\ _ {LA_IS, { "" }, 2, { "binary_action", "binary_action__iteration" }}\ }};\ {LA_NOT, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_OR_EQUAL_0, T_GREATER_OR_EQUAL_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, { "binary_action__iteration", (\ _ {LA_IS, { "" }, 0, { "" }}\ }};\ }\ }\ </pre>
<pre> group_expression = "(" , expression , ")"; </pre>				<pre> group_expression = tokenGROUPEXPRESSIONBEGIN >> expression >> tokenGROUPEXPRESSIONEND; </pre>		<pre> {LA_IS, { "(" , { "group_expression", (\ _ {LA_IS, { "" }, 3, { "(" , "expression", ")" }}\ }};\ }\ </pre>
<pre> expression_or_bind_left_to_right = expression , ["=" , ident , [index_action]]; </pre>				<pre> expression_or_bind_left_to_right = expression >> -(tokenLRBIND >> ident >> -index_action); </pre>		<pre> {LA_IS, { "(" , T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, { "expression_or_bind_left_to_right", (\ {LA_IS, { "" }, 2, { "expression", "bind_to_right__optional" }}\ }};\ }\ </pre>
						<pre> {LA_IS, { T_LRBIND_0 }, { "bind_to_right", (\ _ {LA_IS, { "" }, 3, { T_LRBIND_0, "ident", "index_action__optional" }}\ }};\ }\ </pre>
						<pre> { LA_IS, { T_LRBIND_0 }, { "bind_to_right__optional", (\ _ { LA_IS, { "" }, 1, { "bind_to_right" }}\ }};\ { LA_NOT, { T_LRBIND_0 }, { "bind_to_right__optional", (\ _ { LA_IS, { "" }, 0, { "" }}\ }};\ }\ </pre>
<pre> if_expression = expression; </pre>				<pre> if_expression = SAME_RULE(expression); </pre>		<pre> {LA_IS, { "(" , T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, { "if_expression", (\ {LA_IS, { "" }, 1, { "expression" }}\ }};\ }\ </pre>
<pre> body_for_true = block_statements_in_while_and_if_body; </pre>				<pre> body_for_true = SAME_RULE(block_statements_in_while_and_if_body); </pre>		<pre> {LA_IS, { T_BEGIN_BLOCK_0 }, { "body_for_true", (\ {LA_IS, { "" }, 1, { "block_statements_in_while_and_if_body" }}\ }};\ }\ </pre>
<pre> false_cond_block_without_else = "ELSE" , "IF" , if_expression , body_for_true; </pre>				<pre> false_cond_block_without_else = tokenELSE >> tokenIF >> if_expression >> body_for_true; </pre>		<pre> {LA_IS, { T_ELSE_IF_0 }, { "false_cond_block_without_else", (\ {LA_IS, { "" }, 4, { T_ELSE_IF_0, T_ELSE_IF_1, "if_expression", "body_for_true" }}\ }};\ }\ </pre>
<pre> body_for_false = "ELSE" , block_statements_in_while_and_if_body; </pre>				<pre> body_for_false = tokenELSE >> block_statements_in_while_and_if_body; </pre>		<pre> {LA_IS, { T_ELSE_BLOCK_0 }, { "body_for_false", (\ {LA_IS, { "" }, 2, { T_ELSE_BLOCK_0, "block_statements" }}\ }};\ }\ </pre>
<pre> cond_block = "IF" , if_expression , body_for_true, {false_cond_block_without_else } , [body_for_false]; </pre>				<pre> cond_block = tokenIF >> if_expression >> body_for_true >> *false_cond_block_without_else >> -body_for_false; </pre>		<pre> {LA_IS, { T_IF_0 }, { "cond_block", (\ {LA_IS, { "" }, 5, { T_IF_0, "if_expression", "body_for_true", "false_cond_block_without_else__iteration", "body_for_false__optional" }}\ }};\ }\ </pre>
						<pre> {LA_IS, { T_ELSE_IF_0 }, { "false_cond_block_without_else__iteration", (\ {LA_IS, { T_ELSE_IF_1 }, 2, { "false_cond_block_without_else", "false_cond_block_without_else__iteration" }}\ {LA_NOT, { T_ELSE_IF_1 }, 0, { "" }}\ }};\ {LA_NOT, { T_ELSE_IF_0 }, { "false_cond_block_without_else__iteration", (\ {LA_IS, { "" }, 0, { "" }}\ }};\ }\ </pre>
						<pre> {LA_IS, { T_ELSE_BLOCK_0 }, { "body_for_false__optional", (\ {LA_IS, { "" }, 1, { "body_for_false" }}\ }};\ }\ </pre>

						}}}\n {LA_NOT, { T_ELSE_BLOCK_0 }, { "body_for_false__optional", {\n {LA_IS, {""}, 0, { "" }}\n }}}\n
cond_block__with_optional_bind = cond_block, [tokenLRBIND , ident , [index_action]];				cond_block__with_optional_bind = cond_block >> -(tokenLRBIND >> ident >> -index_action);		{LA_IS, { T_IF_0 }, { "cond_block__with_optional_bind", {\n {LA_IS, {""}, 2, { "cond_block", "bind_to_right__optional" }}\n }}}\n
cycle_begin_expression = expression;				cycle_begin_expression = SAME_RULE(expression);		{LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", " <u>T_IF_0</u> " }, { "cycle_begin_expression", {\n {LA_IS, {""}, 1, { "expression" }}\n }}}\n
cycle_end_expression = expression;				cycle_end_expression = SAME_RULE(expression);		{LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", " <u>T_IF_0</u> " }, { "cycle_end_expression", {\n {LA_IS, {""}, 1, { "expression" }}\n }}}\n
cycle_counter = ident;				cycle_counter = SAME_RULE(ident);		{LA_IS, { "ident_terminal" }, { "cycle_counter", {\n {LA_IS, {""}, 1, { "ident" }}\n }}}\n
cycle_counter_lr_init = cycle_begin_expression , "=: " , cycle_counter;				cycle_counter_lr_init = cycle_begin_expression >> tokenLRBIND >> cycle_counter;		{LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", " <u>T_IF_0</u> " }, { "cycle_counter_lr_init", {\n {LA_IS, {""}, 3, { "cycle_begin_expression", T_LRBIND_0, "cycle_counter" }}\n }}}\n
cycle_counter_init = cycle_counter_lr_init;				cycle_counter_init = SAME_RULE(cycle_counter_lr_init);		{LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", " <u>T_IF_0</u> " }, { "cycle_counter_init", {\n {LA_IS, {""}, 1, { "cycle_counter_lr_init" }}\n }}}\n
cycle_counter_last_value = cycle_end_expression;				cycle_counter_last_value = SAME_RULE(cycle_end_expression);		{LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", " <u>T_IF_0</u> " }, { "cycle_counter_last_value", {\n {LA_IS, {""}, 1, { "cycle_end_expression" }}\n }}}\n
cycle_body = "DO" , {(statement) block_statements};				cycle_body = tokenDO >> (statement block_statements);		{LA_IS, { T_DO_0 }, { "cycle_body", {\n {LA_IS, {""}, 2, { T_DO_0, "statements__or__block_statements" }}\n }}}\n
						{LA_IS, { T_TO_0 }, { "forto_direction", {\n {LA_IS, {""}, 1, { T_TO_0 }}\n }}}\n {LA_IS, { T_DOWNT0_0 }, { "forto_direction", {\n {LA_IS, {""}, 1, { T_DOWNT0_0 }}\n }}}\n
forto_cycle = "FOR" , cycle_counter_init , ("TO" "DOWNT0") , cycle_counter_last_value , cycle_body;				forto_cycle = tokenFOR >> cycle_counter_init >> (tokenTO tokenDOWNT0) >> cycle_counter_last_value >> cycle_body;		{LA_IS, { T_FOR_0 }, { "forto_cycle", {\n {LA_IS, {""}, 5, { T_FOR_0, "cycle_counter_init", "forto_direction", "cycle_counter_last_value", "cycle_body" }}\n }}}\n
				continue_while = SAME_RULE(tokenCONTINUE);		{LA_IS, { "T_CONTINUE_WHILE_0" }, { "continue_while", {\n {LA_IS, {""}, 1, { "T_CONTINUE_WHILE_0" }}\n }}}\n
				break_while = SAME_RULE(tokenBREAK);		{LA_IS, { "T_EXIT_WHILE_0" }, { "break_while", {\n {LA_IS, {""}, 1, { "T_EXIT_WHILE_0" }}\n }}}\n
statement_in_while_and_if_body = statement "CONTINUE" "BREAK";				statement_in_while_and_if_body = statement continue_while break_while;		{LA_IS, { "ident_terminal", "(", T_NOT_0, "unsigned_value_terminal", T_ADD_0, T_SUB_0, T_IF_0, T_FOR_0, T_WHILE_0, T_REPEAT_0, T_GOTO_0, T_INPUT_0, T_OUTPUT_0, T_SEMICOLON_0, { "statement_in_while_and_if_body", {\n {LA_IS, {""}, 1, { "statement" }}\n }}}\n {LA_IS, { T_CONTINUE_WHILE_0 }, { "statement_in_while_and_if_body", {\n {LA_IS, {""}, 1, { "continue_while" }}\n }}}\n {LA_IS, { T_EXIT_WHILE_0 }, { "statement_in_while_and_if_body", {\n {LA_IS, {""}, 1, { "break_while" }}\n }}}\n
block_statements_in_while_and_if_body = "(" , {statement_in_while_and_if_body} , ")";				block_statements_in_while_and_if_body = tokenBEGINBLOCK >> *statement_in_while_and_if_body >> tokenENDBLOCK;		{LA_IS, { T_BEGIN_BLOCK_0 }, { "block_statements_in_while_and_if_body", {\n {LA_IS, {""}, 3, { T_BEGIN_BLOCK_0, "statement_in_while_and_if_body__iteration", T_END_BLOCK_0 }}\n }}}\n
						{LA_IS, { "ident_terminal", "(", T_NOT_0, "unsigned_value_terminal", T_ADD_0, T_SUB_0, T_IF_0, T_FOR_0, T_WHILE_0, T_REPEAT_0, T_GOTO_0, T_INPUT_0, T_OUTPUT_0, T_SEMICOLON_0, T_CONTINUE_WHILE_0, T_EXIT_WHILE_0 }, { "statement_in_while_and_if_body__iteration", {\n {LA_IS, {""}, 2, { "statement_in_while_and_if_body", "statement_in_while_and_if_body__iteration" }}\n }}}\n {LA_NOT, { "ident_terminal", "(", T_NOT_0, "unsigned_value_terminal", T_ADD_0, T_SUB_0, T_IF_0, T_FOR_0, T_WHILE_0, T_REPEAT_0, T_GOTO_0, T_INPUT_0, T_OUTPUT_0, T_SEMICOLON_0, T_CONTINUE_WHILE_0, T_EXIT_WHILE_0 }, { "statement_in_while_and_if_body__iteration", {\n {LA_IS, {""}, 0, { "" }}\n }}}\n
while_cycle_head_expression = expression;				while_cycle_head_expression = SAME_RULE(expression);		{LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", " <u>T_IF_0</u> " }, { "while_cycle_head_expression", {\n {LA_IS, {""}, 1, { "expression" }}\n }}}\n
while_cycle = "WHILE" , while_cycle_head_expression , block_statements_in_while_and_if_body;				while_cycle = tokenWHILE >> while_cycle_head_expression >> block_statements_in_while_and_if_body;		{LA_IS, { T_WHILE_0 }, { "while_cycle", {\n {LA_IS, {""}, 3, { T_WHILE_0, "while_cycle_head_expression", "block_statements_in_while_and_if_body" }}\n }}}\n
repeat_until_cycle_cond = expression;				repeat_until_cycle_cond = SAME_RULE(expression);		{LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", " <u>T_IF_0</u> " }, { "repeat_until_cycle_cond", {\n {LA_IS, {""}, 1, { "expression" }}\n }}}\n
repeat_until_cycle = "REPEAT" , {(statement) block_statements} , "UNTIL" , repeat_until_cycle_cond;				repeat_until_cycle = tokenREPEAT >> (*statement block_statements) >> tokenUNTIL >> repeat_until_cycle_cond;		{LA_IS, { T_REPEAT_0 }, { "repeat_until_cycle", {\n {LA_IS, {""}, 4, { T_REPEAT_0, "statements__or__block_statements", T_UNTIL_0, "repeat_until_cycle_cond" }}\n }}}\n
						{LA_IS, { "ident_terminal", "(", T_NOT_0, "unsigned_value_terminal", T_ADD_0, T_SUB_0, T_IF_0, T_FOR_0, T_WHILE_0, T_REPEAT_0, T_GOTO_0, T_INPUT_0, T_OUTPUT_0, T_SEMICOLON_0, { "statements__or__block_statements", {\n {LA_IS, {""}, 1, { "statement__iteration" }}\n }}}\n {LA_IS, { T_BEGIN_BLOCK_0 }, { "statements__or__block_statements", {\n {LA_IS, {""}, 1, { "block_statements" }}\n }}}\n
input = "GET" , { ident , [index_action] "(" , ident , [index_action] , ")" };				input = tokenGET >> (ident >> -index_action tokenGROUPEXPRESSIONBEGIN >> ident >> -index_action >> tokenGROUPEXPRESSIONEND);		{LA_IS, { T_INPUT_0 }, { "input_rule", {\n {LA_IS, {""}, 2, { T_INPUT_0, "argument_for_input" }}\n }}}\n
						{LA_IS, { "ident_terminal" }, { "argument_for_input", {\n

						{LA_IS, {""}, 2, { "ident", "index_action__optional" }}\ }};\ {LA_IS, { "(" }, { "argument_for_input", {\ {LA_IS, {""}, 4, { "(", "ident", "index_action__optional", ")" }}\ }};\
output = "PUT", expression;				output = tokenPUT >> expression;		{LA_IS, { T_OUTPUT_0 }, { "output_rule", {\ {LA_IS, {""}, 2, {T_OUTPUT_0, "expression"}}\ }};\
statement = expression_or_bind_left_to_right cond_block__with_optional_bind forto_cycle while_cycle repeat_until_cycle labeled_point goto_label input output ";;";				statement = expression_or_bind_left_to_right cond_block__with_optional_bind forto_cycle while_cycle repeat_until_cycle labeled_point goto_label input output tokenSEMICOLON;		{LA_IS, { "ident_terminal" }, { "statement", {\ { LA_IS, { T_COLON_0 }, 1, { "labeled_point" }};\ { LA_NOT, { T_COLON_0 }, 1, { "expression_or_bind_left_to_right" }}\ }};\ {LA_IS, { "(", T_NOT_0, "unsigned_value_terminal", T_ADD_0, T_SUB_0 }, { "statement", {\ { LA_IS, {""}, 1, { "expression_or_bind_left_to_right" }}\ }};\ {LA_IS, { T_IF_0 }, { "statement", {\ {LA_IS, {""}, 1, { "cond_block__with_optional_bind" }}\ }};\ {LA_IS, { T_FOR_0 }, { "statement", {\ {LA_IS, {""}, 1, { "forto_cycle" }}\ }};\ {LA_IS, { T_WHILE_0 }, { "statement", {\ {LA_IS, {""}, 1, { "while_cycle" }}\ }};\ {LA_IS, { T_REPEAT_0 }, { "statement", {\ {LA_IS, {""}, 1, { "repeat_until_cycle" }}\ }};\ {LA_IS, { T_GOTO_0 }, { "statement", {\ {LA_IS, {""}, 1, { "goto_label" }}\ }};\ {LA_IS, { T_INPUT_0 }, { "statement", {\ {LA_IS, {""}, 1, { "input_rule" }}\ }};\ {LA_IS, { T_OUTPUT_0 }, { "statement", {\ {LA_IS, {""}, 1, { "output_rule" }}\ }};\ {LA_IS, { T_SEMICOLON_0 }, { "statement", {\ {LA_IS, {""}, 1, { ";" }}\ }};\
						{ LA_IS, { "ident_terminal", "(", T_NOT_0, "unsigned_value_terminal", T_ADD_0, T_SUB_0, T_IF_0, T_FOR_0, T_WHILE_0, T_REPEAT_0, T_GOTO_0, T_INPUT_0, T_OUTPUT_0, T_SEMICOLON_0 }, { "statement__iteration", {\ { LA_IS, {""}, 2, { "statement", "statement__iteration" }}\ }};\ { LA_NOT, { "ident_terminal", "(", T_NOT_0, "unsigned_value_terminal", T_ADD_0, T_SUB_0, T_IF_0, T_FOR_0, T_WHILE_0, T_REPEAT_0, T_GOTO_0, T_INPUT_0, T_OUTPUT_0, T_SEMICOLON_0 }, { "statement__iteration", {\ { LA_IS, {""}, 0, { "" }}\ }};\
block_statements = "{", {statement}, "}";				block_statements = tokenBEGINBLOCK >> *statement >> tokenENDBLOCK;		{ LA_IS, { T_BEGIN_BLOCK_0 }, { "block_statements", {\ { LA_IS, {""}, 3, { T_BEGIN_BLOCK_0, "statement__iteration", T_END_BLOCK_0 }}\ }};\
						{LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, { "expression_optional", {\ {LA_IS, {""}, 1, { "expression" }}\ }};\ {LA_NOT, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, { "expression_optional", {\ {LA_IS, {""}, 0, { "" }}\ }};\
program = "NAME", program_name, ",", "BODY", "DATA", [declaration], ",", {statement}, "END";				program = BOUNDARIES >> tokenNAME >> program_name >> tokenSEMICOLON >> tokenBODY >> tokenDATA >> {-declaration} >> tokenSEMICOLON >> *statement >> tokenEND;		{ LA_IS, { T_NAME_0 }, { "program_rule", {\ { LA_IS, {""}, 9, { T_NAME_0, "program_name", T_SEMICOLON_0, T_BODY_0, T_DATA_0, "declaration__optional", T_SEMICOLON_0, "statement__iteration", T_END_0 }}\ }};\
						{ LA_IS, { T_DATA_TYPE_0 }, { "declaration__optional", {\ { LA_IS, {""}, 1, { "declaration" }}\ }};\ { LA_NOT, { T_DATA_TYPE_0 }, { "declaration__optional", {\ { LA_IS, {""}, 0, { "" }}\ }};\
digit = "0" "1" "2" "3" "4" "5" "6" "7" "8" "9";				digit = digit_0 digit_1 digit_2 digit_3 digit_4 digit_5 digit_6 digit_7 digit_8 digit_9;		\
non_zero_digit = "1" "2" "3" "4" "5" "6" "7" "8" "9";				non_zero_digit = digit_1 digit_2 digit_3 digit_4 digit_5 digit_6 digit_7 digit_8 digit_9;		\
unsigned_value = (non_zero_digit, {digit}) "0";				unsigned_value = (non_zero_digit >> *digit digit_0) >> BOUNDARIES;		\
value = [sign], unsigned_value;				value = -sign >> unsigned_value >> BOUNDARIES;		{LA_IS, { "unsigned_value_terminal" }, { "unsigned_value", {\ {LA_IS, {""}, 1, { "unsigned_value_terminal" }}\ }};\ {LA_IS, { "unsigned_value_terminal", T_ADD_0, T_SUB_0 }, { "value", {\ {LA_IS, {""}, 2, { "sign_optional", "unsigned_value" }}\ }};\
						{LA_IS, { T_ADD_0, T_SUB_0 }, { "sign_optional", {\ {LA_IS, {""}, 1, { "sign" }}\ }};\ {LA_NOT, { T_ADD_0, T_SUB_0 }, { "sign_optional", {\ {LA_IS, {""}, 0, { "" }}\ }};\
letter_in_lower_case = "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s" "t" "u" "v" "w" "x" "y" "z";				letter_in_lower_case = a b c d e f g h i j k l m n o p q r s t u v w x y z;		\
letter_in_upper_case = "A" "B" "C" "D" "E" "F" "G" "H" "I" "J" "K" "L" "M" "N" "O" "P" "Q" "R" "S" "T" "U" "V" "W" "X" "Y" "Z";				letter_in_upper_case = A B C D E F G H I J K L M N O P Q R S T U V W X Y Z;		\
ident = "_", letter_in_upper_case, letter_in_upper_case, letter_in_upper_case, letter_in_upper_case, letter_in_upper_case, letter_in_upper_case, letter_in_upper_case;				ident = tokenUNDERSCORE >> letter_in_upper_case >> letter_in_upper_case >> letter_in_upper_case >> letter_in_upper_case >> letter_in_upper_case >> letter_in_upper_case >> letter_in_upper_case >> STRICT_BOUNDARIES;		{LA_IS, { "ident_terminal" }, { "ident", {\ {LA_IS, {""}, 1, { "ident_terminal" }}\ }};\
sign = "+" "-";				sign = sign_plus sign_minus;		{LA_IS, { T_ADD_0 }, { "sign", {\ {LA_IS, {""}, 1, { "sign_plus" }}\ }};\ {LA_IS, { T_SUB_0 }, { "sign", {\ {LA_IS, {""}, 1, { "sign_minus" } } }\ }};\
				sign_plus = SAME_RULE(tokenPLUS);		{LA_IS, { T_ADD_0 }, { "sign_plus", {\ {LA_IS, {""}, 1, {T_ADD_0}}\ }};\

[illegible]

						\
						\
				STRICT_BOUNDARIES = (BOUNDARY >> *(BOUNDARY)) (!qi::alpha qi::char_("_ "));		\
				BOUNDARIES = (BOUNDARY >> *(BOUNDARY) NO_BOUNDARY);		\
				BOUNDARY = BOUNDARY_SPACE BOUNDARY_TAB BOUNDARY_CARRIAGE_RETURN BOUNDARY_LINE_FEED BOUNDARY_NULL;		\
				BOUNDARY_SPACE = " ";		\
				BOUNDARY_TAB = "\t";		\
				BOUNDARY_CARRIAGE_RETURN = "\r";		\
				BOUNDARY_LINE_FEED = "\n";		\
				BOUNDARY_NULL = "\0";		\
				NO_BOUNDARY = "";		\
				tokenUNDERSCORE = "_";		\
				A = "A";		\
				B = "B";		\
				C = "C";		\
				D = "D";		\
				E = "E";		\
				F = "F";		\
				G = "G";		\
				H = "H";		\
				I = "I";		\
				J = "J";		\
				K = "K";		\
				L = "L";		\
				M = "M";		\
				N = "N";		\
				O = "O";		\
				P = "P";		\
				Q = "Q";		\
				R = "R";		\
				S = "S";		\
				T = "T";		\
				U = "U";		\
				V = "V";		\
				W = "W";		\
				X = "X";		\
				Y = "Y";		\
				Z = "Z";		\
				a = "a";		\
				b = "b";		\
				c = "c";		\
				d = "d";		\
				e = "e";		\
				f = "f";		\
				g = "g";		\
				h = "h";		\
				i = "i";		\
				j = "j";		\
				k = "k";		\
				l = "l";		\
				m = "m";		\
				n = "n";		\
				o = "o";		\
				p = "p";		\
				q = "q";		\
				r = "r";		\

				s = "s";		\
				t = "t";		\
				u = "u";		\
				v = "v";		\
				w = "w";		\
				x = "x";		\
				y = "y";		\
				z = "z";		\
						\ \ \ \ { LA_IS, { T_NAME_0 }, { "program____part1", {\ { LA_IS, { "" }, 7, { T_NAME_0, "program_name", T_SEMICOLON_0, T_BODY_0, T_DATA_0, "declaration_optional", T_SEMICOLON_0 }}\ }};\ \ };\ "program_rule"